Exp No: 14	**Exploring pro-code extension of a Zoho Creator application using Zoho Catalyst / custom functions.**

AIM:

To explore the pro-code extension of a Zoho Creator application by writing a custom function in Deluge and by creating a serverless function in Zoho Catalyst, and to call it from the Creator application for the work that is not easy with the low-code builders.

ALGORITHM / PROCEDURE:

When the logic of an application becomes too big for the drag and drop builders, the developer can extend it with the real code. In Zoho Creator this can be done in two ways. A **custom function** is a reusable Deluge function which is written once and called from many workflows. **Zoho Catalyst** is the serverless platform of Zoho in which the functions can be written in languages like Node.js or Java and deployed on the cloud. The Creator application then calls the Catalyst function by an API call using *invokeUrl*. In this experiment a bill amount calculation (GST and total) is done by both the ways.

1. Create an application named *Billing* with a form *Invoice* having the fields Customer, Amount, GST and Total.
2. **Custom function:** go to **Workflow** → **Functions** and create a new function *calculateGST* with the parameter amount (Script 1). Save it.
3. Open the form Invoice → workflow → **On Validate** and call the function (Script 3, first part). Submit a record and check that the GST and Total fields are filled.
4. **Catalyst:** log in to Zoho Catalyst and create a new project named *BillingExt*. Install the Catalyst CLI on the computer, log in, and create the function using the commands given below (Script 2).
5. Write the Node.js code of the function (Script 4), which receives the amount and returns the GST and the total in JSON. Test it locally with **catalyst serve** and then deploy it with **catalyst deploy**.
6. Copy the URL of the deployed function from the Catalyst console. In Creator create a **connection** (Microservices → Connections) for the authorisation of the Catalyst API, if the function is not public.
7. In the workflow of the form call the Catalyst function using **invokeUrl** (Script 3, second part) and store the values from the response in the fields.
8. Submit the records again and check the output. Open the **Logs** of the Catalyst function to see that the request came from Creator and note the execution time.
9. Compare the two methods (custom function and Catalyst) and note down the observations.

Script 1 - Deluge custom function in Creator

```
float calculateGST(float amount)
{
    return amount * 0.18;
}
```

Script 2 - Catalyst CLI commands (run in the terminal)

```
npm install -g zcatalyst-cli
catalyst login
catalyst init
catalyst functions:add
catalyst serve
catalyst deploy
```

Script 3 - On Validate of the form Invoice (Deluge)

```
// method 1 : call the custom function
input.GST = thisapp.calculateGST(input.Amount);
input.Total = input.Amount + input.GST;

// method 2 : call the Catalyst function using the connection
resp = invokeUrl
[
    url : "https://<project-domain>/server/gst_function/execute?amount=" + input.Amount
    type : GET
    connection : "catalyst_conn"
];
info resp;
```

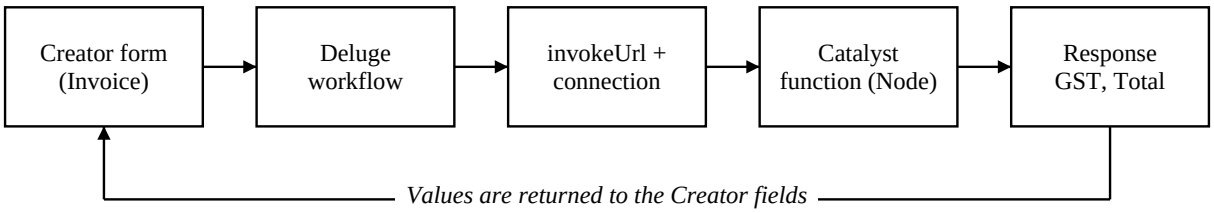
Script 4 - Catalyst function (Node.js, Basic I/O) gst_function/index.js

```
module.exports = (context, basicIO) => {
    const amount = parseFloat(basicIO.getArgument("amount"));
    const gst = amount * 0.18;
    basicIO.write(JSON.stringify({ gst: gst, total: amount + gst }));
    context.close();
};
```

Note: the function name, the project domain in the URL and the way of authorisation come from the Catalyst project of the user, so they should be replaced with the actual values. A Catalyst function is useful for the heavy calculations, the use of the code libraries and the work that takes more time than the Deluge limits.

DIAGRAM:

Fig 1: Pro-code extension of Zoho Creator using Zoho Catalyst



RESULT:

Thus the pro-code extension of a Zoho Creator application was explored. The GST and the total amount were calculated using a Deluge custom function and also by a Node.js function deployed in Zoho Catalyst and called from Creator using invokeUrl, which shows how the low-code application can be extended with the real code when needed.